

Final Paper

- **Title:** *Foodifye* E-commerce of Canteen
- **Problem Statement / Rationale** - Provides a compelling problem or need that the project addresses. Shows real-world or classroom relevance. wip gavins part finish if y'all wanna tho

In many school canteens, including the PSHS CALABARZONRC canteen, the inefficiency of ordering food is one of the main problems that school cafeterias face. A lot of scholars like us are mostly relying on manual ordering, creating a lot of problems for the cafeteria staff to handle, including long lines, inaccurate calculations of the total cost, and possible miscommunication. Although the project we made aimed to fix all those problems using a Python-based program that was specifically made to address this issue. With this program, there would be no inaccurate calculations, little to no misunderstandings, and shorter lines. Most importantly, one of the main issues of food ordering would be fixed.

- **Project Overview and Relevance** - Clear, concise description of the project, with strong relevance to Python, database, and data analysis concepts.

The project we made is a Python-based canteen ordering system to improve the efficiency of food ordering on campus. The program shall ask the user to input their campus email, choose meal options such as rice, the cafeteria's main dishes, and drinks, and can also calculate the total price of what they ordered.

Our program uses Python programming concepts such as loops, lists, and dictionaries so that it can manage what the student inputs and the data of the menu itself. The features that we used will ensure proper validation, so that the user will experience no problems during the ordering process.

For the data analysis, the system processes the options that were selected by the user so that the program can compute the total price. This shows us that data can be analyzed and handled by the program in real time. We aimed to exactly replicate real-world food ordering, where systems shall process the transactions made quickly and accurately.

To conclude, the project we made shows us how Python can be used to develop systems that deal with managing and processing data, along with interacting with the user. Because of this project, we learned how to apply the fundamental programming concepts, such as input validation, loops, and data structures, to create a functional and accurate food ordering system.

- **Objectives** - Well-stated, SMART objectives aligned with project goals.

- To develop an ordering system using Python
 - Allow users to log in using a campus-based email format (e.g., juandelacruz@cbzrc.pshs.edu.ph) and validate their input correctly.
- To implement input validation and error handling
 - With the use of loops and conditional statements, we can ensure that users will enter a valid campus-based email format, the menu, and what they want to order.
- To create a program for the menu that will allow the user to
 - Select rice options
 - Select main dishes
 - Select a drink (optional)
- To calculate the total cost automatically
 - Based on the selected food, using the dictionaries for pricing
- To replicate real-world food ordering
 - With the use of options such as dine-in/takeout and the payment method
- **Methodology / Technical Approach** - Clear and detailed plan on how to use Python, JSON/Flask, CRUD, and data analysis. Shows good understanding.

This program develops a Canteen E-Commerce system using Python, JSON, CRUD, and data analysis. The system is designed to allow students to log-in, select food items, and automatically calculate the total price of their order. Python is the programming language used to create the program and JSON is used to store and manage data in a readable format.

System Design

This system uses a simple Input-Process-Output(IPO) model:

Input- User enters their log-in details(username and password), order type(order or pre-order), and food choices.

Process- The system checks inputs, calculates prices, and manages orders.

Output- The system displays the total price and order summary.

Python Implementation

Python is used to program the system because it is an efficient and easy to understand programming language. The program uses basic Python functions such as lists, dictionaries, and loops. Lists are used to store menu options(rice, main dish, drinks) while dictionaries are used to store the prices for each item. To control program flow and validate input, we use loops. If-else and while statements are widely used throughout the code to help the user pick their order. To prevent errors, the system uses input validation statements such as try-except blocks. This makes the program organized and easy to maintain.

Data Storage using JSON

JSON is more suitable than Flask for this program because it is lightweight, readable, and compatible with Python. Although the current program does not save data, its structure allows easy integration with JSON for data storage. Each order can be organized into a JSON format that contains the user's email, campus, selected items, and total price.

CRUD Operations

This system supports basic CRUD operations. A new order is created when a user completes the ordering process. The order summary will be shown to the user. The program also allows you to update or change something in your order before finalizing. And if you wish to cancel your order, it can always be deleted.

Create- User places an order

Read- User can view order summary

Update- User can change their order before finalizing

Delete- User can cancel their order

Data Analysis

The system also uses basic data analysis using the data collected from orders. Since the program stores food items in a list, it's possible to identify the most ordered food items. The program can also use the data gathered to see the sales performance. It tracks monthly sales, stock levels, and overall profit.

Program Flow

The program is divided into several sections including log-in, order selection, food selection, quantity input, and order summary. Each section is controlled by loops to ensure valid inputs before moving forward. At the end, you can see the summary of your order before finalizing. This structure helps maintain an organized and smooth flow throughout the program.

- **Flowchart** – Provides a detailed flowchart of each component or Module.

<https://onedrive.live.com/?redeem=aHR0cHM6Ly8xZHJ2Lm1zL3UvYy8xNEMwMzc4NEU4Mzg2QURCL0IRQI9mOHhFWnl2S1JxeEhZM0QweE94cUFmbU4tLU9lUk5iMDJSaGpMV3lTTk5zP2U9UTZHV0FRJmZiY2xpZD1Jd1kyeGphd1JRWDAXbGVIUnVBMkZsYIFJeE1RQnpjblJqQm1Gd2NGOXBaQUV3QUFFZWVQVEVyTUZuWmtncUluVWpoT0NYeHBQbEhkTnYyQ1YzcXg4akVTOFdFYnpVc1UwelZkcGwcziMWIFvX2FlbV9YTnR5Vzh5RWtPQXJVNzhBNlV1Rndn&cid=14C03784E8386ADB&id=14C03784E8386ADB%21s4>

[4cc7f7fbe66468aac476370f4c4ec6a&parId=14C03784E8386ADB%21s3ae857fbf9df4421872177fcdecd324a&o=OneUp](https://api.canteen.ph/4cc7f7fbe66468aac476370f4c4ec6a&parId=14C03784E8386ADB%21s3ae857fbf9df4421872177fcdecd324a&o=OneUp)

(sobrang laki maam, nilagay nlng sa one drive :'))

- **Program** – Provides screenshots of code and output.

CODE & OUTPUT

```
TOTAL PRICE=0
CAMPUS=["CBZRC", "IRC", "WVC", "CLC",
        "CVC", "CMC", "MC", "ZRC",
        "EVC", "CVISC", "DRC", "MRC",
        "BRC", "ZPRC", "NMC", "SRC", "CARC"]#Campus abbreviations
RICEPRICE={"Whole": 50, "Half":25, "None":0}#Rice prices
MAINSPRICE={"Seafood":40, "Beef":35,
            "Vegetables":35, "Pork":25, "Chicken":30}#Main prices
SHAKE_FLAVORS=["Banana","Straberry","Blueberry","Muscat grape",
               "Red grapes","Dragon fruit","Mango","Watermelon"
               "Buko", "Guyabano","Apple", "Lychee",
               "Pineapple"]#Shake flavors
DAIRY=["Milo", "Milk", "Yakult"]
JUICE_FLAVORS=['Pink lemonade', 'Blue lemonade', 'Iced tea', 'Cucumber', 'C2', 'Mogu mogu', 'Pineapple'
               'Apple', 'Buko', 'Lemonade', 'Orange', 'Blueberry', 'Strawberry', 'Lemon', ]#Juice options

MAIN=[]
RICE=[]
DRINK=[]

counter = 2

def address(email_name, abr):
    return f"{email_name}@{abr.lower()}.pshs.edu.ph"

print("""
===FOODIFYE | E-COMMERCE CANTEEN===""")

pick = str(input("""

Welcome, dear user!

Sign-in? (Yes/No): """).title()
while pick not in ["Yes", "No"]:
    print("""
---Invalid choice, kindly try again---""")
    pick = str(input("""

Welcome, dear user!

Sign-in? (Yes/No): """).title() # Corrected method
if pick == "Yes":
    print("""
---Proceed with the following instructions---""")
```

```

"""

#SIGNING IN SEGMENT#
while True:
    try:
        name=str(input("Enter your email name (Ex. jcdeguzman): "))
        campus=str(input(f"Enter your campus abbreviation (Ex. CBZRC): ")).upper()
        while campus not in CAMPUS:
            print("""
---Campus address not found, kindly try again---
""")
            campus=str(input(f"Enter your campus abbreviation: ")).upper() #Error handling no.1 for campus address
        psswr=str(input("""
Enter your password: """))
        print("""
---Password integrated to log-in---
""")
        email=address(name, campus)
        print(f""""Welcome, {email}!

---
""")
        break
    except ValueError:
        print(f""""
---Error in values entered, kindly try again---
""")

#HANDLING AND MODE OF RECEIVE SEGMENT#
while True:
    try:
        choice=input("Order OR Pre-Order? (If 'Pre-Order, please enter 'preorder'): ").title()
        while choice not in ["Order", "Pre order", "Preorder"]:#Error handling no.2 for order handling
            print("""
---Invalid option, kindly try again---
""")
        choice=input("Order OR PreOrder?: ").title()

```

```

        mode=input("Takeout OR Dine-in? (If 'Dine-in', please enter 'dinein'): ").title()
        while mode not in ["Take-out", "Takeout", "Take out", "Dinein"]:
            print("""
---Invalid option, kindly try again---
""")
            mode=input("Takeout OR Dinein?: ").title()
            break

#END OF SEGMENT#
except ValueError:
    print(f"""---Error in values entered
, kindly try again---""")

#ORDERING SEGMENT#
while True:
    try:
        if mode=="Dinein":#Plate section for dining-in
            print("""
---You have a limit of three plates---
""")
            plate_counter=int(input("How many plates?: "))
            while plate_counter>3 or plate_counter<0:
                if plate_counter>3:
                    print("""
---Maximum of 3 plates, kindly insert another input---
""")
                    plate_counter=int(input("How many plates?: "))
                else:
                    print("""
---Invalid input, kindly try again---
""")
                    plate_counter=int(input("How many plates?: "))
            break
        else:#Takout section for boxes
            print("""
---You have a limit of three boxes---
""")
            box_counter=int(input("How many boxes? (Do not input negative integers): "))
            match box_counter:
                case _ if box_counter>3:
                    while box_counter>3:

```

```

        print("""
---Invalid input, kindly try again---
""")
        box_counter=int(input("How many boxes?: "))
        break
    except ValueError:
        print(f"""
---Error in values entered, kindly try again---""")

while True:
    try:
        print("""
---
""")
        rice_response=str(input("Would you like some rice? (Yes/No): ")).title()
        while rice_response!="Yes" and rice_response!="No":#Error handling
            print("""
---Kindly try again---
""")
            rice_response=str(input("Would you like some rice? (Yes/No): ")).title()

        if rice_response=="Yes":
            r=counter
            for i in range(0, r):
                rice_pick=str(input("Choose either Whole, Half, or None for rice (Enter twice): ")).title()
                while rice_pick not in ["Whole", "Half", "None"]:
                    print("""
---Choice does not exist, kindly try again---
""")
                    rice_pick=str(input("Choose either Whole, Half, or None for rice: ")).title()
                continue
            TOTAL_PRICE = RICEPRICE[rice_pick]+TOTAL_PRICE
            c=2*counter#Display choices remaining, dependant on no. of plates ordered

            for i in range(0,c):
                print(f"""
---Enter your main dishes. You have {c} options left. Enter 0 to for none---
( Seafood = 40 / Beef = 35 / Vegetables = 35 / Pork = 25 / Chicken = 30 )
""")
                main_dish=str(input("Enter a Main dish or dishes to go: ")).title()

```

```

        if main_dish=='0':
            break

        while main_dish not in ["Chicken","Pork","Beef","Vegetables","Seafood"]:
            print("""
---Choice does not exist, kindly try again---
""")

            main_dish=str(input("Enter a Main dish to go: ")).title()

            TOTAL_PRICE=TOTAL_PRICE+MAINSPRICE[main_dish]
            MAIN.append(main_dish)
            c-=1

        break
#END OF SEGMENT#
    except ValueError:
        print(f"""
---Error in values entered, kindly try again---
""")

while True:
    try:
        print("""
---
""")

        drink_response=input("Would you like drinks (Yes/No)?: ").title()
        while drink_response not in ["Yes", "No"]:
            print("""
---Choice does not exist, kindly try again---
""")

            drink_response = input("Would you like drinks (Yes/No)?: ").title()

        if drink_response=="Yes":
            print("""
---Limit of 10 drinks---
""")

            drink_counter = int(input("How many drinks do you want?: "))
            while drink_counter < 0 or drink_counter > 10:
                print("Kindly try again")
                print("""
---Maximum of 10 drinks, minimum of 0---""")

            drink_counter=int(input("How many do you want?: ").title())

```



```

print(f"""
---DRINKS---

AVAILABLE JUICES:
{JUICE_FLAVORS}
""")

print(f"""AVAILABLE SHAKES:

{SHAKE_FLAVORS}
""")

print(f"""AVAILABLE DAIRY:

{DAIRY}""")

e = drink_counter
for i in range(0,e):
    print(f"""
---Enter some drinks to go. You have {e} choice/s left. Enter 0 to exit---
""")

    drink=str(input("Would you like a Shake, Juice, or Dairy?: ").title())
    while drink not in ["Shake","Juice","Dairy", "0"]:
        print(f"""
---Kindly try again
""")

        drink=str(input("Would you like a Shake, Juice, or Dairy?: ").title())

    if drink=="0":
        break

    elif drink=="Shake":
        TOTAL_PRICE=TOTAL_PRICE+20
        drink_pick=input(f"""
---Look at the choices above---

Chosen shake type: """).title()

        while drink_pick not in SHAKE_FLAVORS:
            ("""
---Option not found, kindly try again---
""")

            drink_pick=input(f"""
---Look at the choices above---

Chosen shake type: """).title()

    elif drink=="Juice":
        TOTAL_PRICE=TOTAL_PRICE+25
        drink_pick=str(input(f"""

```

```

---Look at the choices above---

Chosen shake type: "").title()
                                while drink_pick not in SHAKE_FLAVORS:
                                    ("""
---Option not found, kindly try again---
""")
                                drink_pick=input(f"""

---Look at the choices above---

Chosen shake type: "").title()

                                elif drink=="Juice":
                                    TOTAL_PRICE=TOTAL_PRICE+25
                                    drink_pick=str(input(f"""

---Look at the choices above---|
Chosen juice type: "").title()
                                while drink_pick not in JUICE_FLAVORS:
                                    print("""
---Option not found, kindly try again---
""")
                                drink_pick=str(input(f"""

---Look at the choices above---

Chosen juice type: "").title()
                                else:
                                    TOTAL_PRICE=TOTAL_PRICE+27
                                    drink_pick=str(input(f"""

---Look at the choices above---

Chosen dairy type: "").title()
                                while drink_pick not in DAIRY:
                                    print("""
---Option not found, kindly try again---
""")
                                drink_pick=str(input(f"""

---Look at the choices above---

Chosen dairy type: "").title()
                                DRINK.append(drink_pick)
                                e = 1

```

```

        break
    except ValueError:
        print(f"""
---Invalid input. Try again---
""")

#END OF SEGMENT#

#OUTPUT SEGMENT#
print("""---

===FINAL RECEIPT===
""")
print(f"""EMAIL: {email}
CHOICE: {choice}
PLATES/BOXES ORDERED: {counter}""")
if rice_response == "No":
    print(f"RICE: NONE")
else:
    print(f"RICE: {rice_pick} rice")
print(f"MAIN: {MAIN}")
print(f"NO. OF DRINKS ORDERED: {drink_counter}")

if drink_response == "No":
    print(f"DRINK/S: NONE")
else:
    print(f"DRINK/S: {drink_pick}")

print(f"TOTAL PRICE: ₱{TOTAL_PRICE}")

print("""

===THANK YOU FOR USING FOODIFYE FOR YOUR PURCHASES===

===GOODBYE FELLOW CUSROMER!!===

""")

input("Press anything to exit...")

```

===FOODIFYE | E-COMMERCE CANTEEN===

Welcome, dear user!

Sign-in? (Yes/No): yes

|

---Proceed with the following instructions---

Enter your email name (Ex. jcdeguzman): rvabella

Enter your campus abbreviation (Ex. CBZRC): cbzrc

Enter your password: test123

---Password integrated to log-in---

Welcome, rvabella@cbzrc.pshs.edu.ph!

Order OR Pre-Order? (If 'Pre-Order, please enter 'preorder'): preorder

Takeout OR Dine-in? (If 'Dine-in', please enter 'dinein'): dinein

---You have a limit of three plates---

How many plates?: 1

```

---

Would you like some rice? (Yes/No): yes
Choose either Whole, Half, or None for rice (Enter twice): half
Choose either Whole, Half, or None for rice (Enter twice): half

---Enter your main dishes. You have 4 options left. Enter 0 to for none---
( Seafood = 40 / Beef = 35 / Vegetables = 35 / Pork = 25 / Chicken = 30 )

Enter a Main dish or dishes to go: chicken'

---Choice does not exist, kindly try again---

Enter a Main dish to go: chicken

---Enter your main dishes. You have 3 options left. Enter 0 to for none---
( Seafood = 40 / Beef = 35 / Vegetables = 35 / Pork = 25 / Chicken = 30 )

Enter a Main dish or dishes to go: 0

---

Would you like drinks (Yes/No)? : yes

---Limit of 10 drinks---

How many drinks do you want?: 1

```

```

How many drinks do you want?: 1

---DRINKS---

AVAILABLE JUICES:
['Pink lemonade', 'Blue lemonade', 'Iced tea', 'Cucumber', 'C2', 'Mogu mogu', 'PineappleApple', 'Buko', 'Lemonade', 'Orange', 'Blueberry', 'Strawberry', 'Lemon']

AVAILABLE SHAKES:
['Banana', 'Straberry', 'Blueberry', 'Muscat grape', 'Red grapes', 'Dragon fruit', 'Mango', 'WatermelonBuko', 'Guyabano', 'Apple', 'Lychee', 'Pineapple']

AVAILABLE DAIRY:
['Milo', 'Milk', 'Yakult']

---Enter some drinks to go. You have 1 choice/s left. Enter 0 to exit---

Would you like a Shake, Juice, or Dairy?: dairy

---Look at the choices above---

Chosen dairy type: milk
---

===FINAL RECEIPT===

EMAIL: rvabella@cbzrc.pshs.edu.ph
CHOICE: Preorder
PLATES/BOXES ORDERED: 2
RICE: Half rice
MAIN: ['Chicken']
NO. OF DRINKS ORDERED: 1
DRINK/S: Milk
TOTAL PRICE: ₱107

===THANK YOU FOR USING FOODIFYE FOR YOUR PURCHASES===

===GOODBYE FELLOW CUSROMER!!===

Press anything to exit...

```

- **Team Roles** – in table format, from first quarter to fourth quarter

Member Names	Q1	Q2	Q3	Q4
Abella, Rysa	Project Checker Editor	Project Checker	Editor Programmer	Programmer Project Checker
Bacolod, Sofia	Logic Planner	Logic Planner Editor	Programmer Editor	Editor Methodology Writer
Calleja, Aiden	Programmer Project Checker	Programmer Editor	Programmer	Programmer Project Checker
Llagas, Emil	Programmer Logic Planner	Logic Planner	Logic Planner Project Decomposition Editor	Project Decomposition Editor Flowchart Maker
Perez, Gavin	Editor Programmer	Editor Project Checker	Editor	Editor Logic Planner

- **References / Background Research** - Includes 2 or more relevant sources in APA format; demonstrates background research.

- *JSON - JavaScript* | MDN. (2025, November 20).

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON?fbclid=IwY2xjawRQWutleHRuA2FlbQIxMQBzcnRjBmFwcF9pZAEwAAEeZoUYBILdk3pWZmPMA57khszr_rDu7FBIiOilWZtFJT9YWPqnYiHiv0j4Q0_aem_2F84yhMJgN_JYkygu9YkA

- *Think Topics* | IBM. (n.d.). <https://www.ibm.com/think/topics>
- (srry ma'am if kaunti lng, ginamit nman as references mostly ay ung readme, actual code, and stock knowledge sa paggawa, ehehe.)

Final Readme file on a separate page